
Sistemas Operacionais II

Entrega 4

Anderson Simioni Assunção (23150566)

Bruno Pavani Bertolino (23100464)

Gustavo Fukushima de Salles (21250090)

Julia Fischer Gazolla (23250586)

Classes modificadas para P4

```
C vehicle_sensor_codes.h x
src > C vehicle_sensor_codes.h > ...
1  #pragma once
2  #include <stdint>
3
4  // Vehicle data type codes inspired by IEEE 1
5  // Each code uniquely identifies a sensor or
6  enum class VehicleDataType : uint16_t {
7      ENGINE_RPM          = 0x0001,
8      VEHICLE_SPEED       = 0x0002,
9      ENGINE_TEMPERATURE  = 0x0003,
10     FUEL_LEVEL           = 0x0004,
11     BATTERY_VOLTAGE      = 0x0005,
12     OIL_PRESSURE         = 0x0006,
13
14     GPS_LOCATION         = 0x0007,
15     AMBIENT_TEMPERATURE  = 0x0008,
16 };
```

- Makefile, para encerramento das máquinas virtuais
 - Novas classes: publish_subscriber.h e vehicle_sensor_codes.h
 - Car component ganha os elementos necessários para publish-subscribe
-

Melhorias gerais para P4

- Os veículos encerram por timeout
- O código gera um arquivo de logs
- As latências são calculadas por script

```
vm_0_65535.log x
logs > vm_0 > vm_0_65535.log

1  [RECV VM=1 PORT=65535 T=1760928374206121 ID=0]
2
3  [SEND VM=0 PORT=65535 T=1760928374219881 ID=0]
4
5  [RECV VM=1 PORT=65535 T=1760928384213031 ID=2]
6
7  [SEND VM=0 PORT=65535 T=1760928384217133 ID=1]
8
9  [RECV VM=1 PORT=65535 T=1760928394228014 ID=4]
10
11 [SEND VM=0 PORT=65535 T=1760928394233755 ID=2]
12
13 [RECV VM=1 PORT=65535 T=1760928404232033 ID=6]
```

Exemplo de envio e recebimento - ETH e SHM

```
NIC sending through SHM
serializing SHM Ethernet Frame..
===== FRAME =====
SRC: 52:54:00:12:34:04
DST: 52:54:00:12:34:04
PROTO: 0x0123
SIZE: 135 bytes
PAYLOAD (texto): ..f...TS=1760926635091930 VM=4 PORT=65535 ID=11
=====
sending SHM Ethernet Frame..
packet: RT
sending message: RT
locking bus..
writing message..
unlocking bus..
[RECV VM=4 PORT=65535 T=1760926635091930 VM=4 PORT=65535 ID=11]
```

```
[DEBUG] Gateway: Receiving message, origin: 1037 103
[DEBUG] Received payload from Ethernet, forwarding internally
[DEBUG] Fanning out message [TS=1760926635035674 VM=4 PORT=65535 ID=9 TS=1760926635030145 VM=4 PORT=102 ID=3 brake!]
[SEND VM=4 PORT=65535 T=1760926635045907 ID=10]

NIC sending through SHM
serializing SHM Ethernet Frame..
===== FRAME =====
SRC: 52:54:00:12:34:04
DST: 52:54:00:12:34:04
PROTO: 0x0123
SIZE: 134 bytes
PAYLOAD (texto): ..f...TS=1760926635045907 VM=4 PORT=65535 ID=10 TS=1760926635035674 VM=4 PORT=65535 ID=9 TS=1760926635030145 VM=4 PORT=102 ID=3 brake!
=====
sending SHM Ethernet Frame..
packet: RT
sending message: RT
locking bus..
writing message..
unlocking bus..
[RECV VM=4 PORT=65535 T=1760926635045907 VM=4 PORT=65535 ID=10 TS=1760926635035674 VM=4 PORT=65535 ID=9 TS=1760926635030145 VM=4 PORT=102 ID=3 brake!]
```

```
NIC sending through ETHERNET
54 53 3D 31 37 36 30 39 32 36 37 39 35 34 35 30 36 33 30 20 56 4D 3D 31 20 50 4F 52 54 3D 36 35 35 33
[DEBUG] Forwarding to port [65535]
[RECV VM=1 PORT=65535 T=1760926795455991 ID=67]
```

Publish and Subscriber

```
static bool digest_subs(const Rx& rx) {
    std::string s(rx.payload.begin(), rx.payload.end());
    size_t p1 = s.find("SUBS=");
    size_t p2 = s.find("SUBS_TYPE=");
    size_t p3 = s.find("SUBS_PERIOD=");
    if (p1 == std::string::npos || p2 == std::string::npos || p3 == std::string::npos) return false;

    size_t e1 = s.find(' ', p1); if (e1 == std::string::npos) e1 = s.size();
    size_t e2 = s.find(' ', p2); if (e2 == std::string::npos) e2 = s.size();
    size_t e3 = s.find(' ', p3); if (e3 == std::string::npos) e3 = s.size();

    std::string addr = s.substr(p1 + 5, e1 - (p1 + 5));
    VehicleDataType dtype = std::stoi(s.substr(p2 + 10, e2 - (p2 + 10)));
    u64 period = std::stoull(s.substr(p3 + 12, e3 - (p3 + 12)));

    if(this->_pubSub.supports(dtype)) this->_pubSub.subscribe(dtype, addr, period);

    return true;
}
```

Publish and Subscriber

You, 53 seconds ago | 1 author (You)

```
class PublishSubscriber {
```

```
public:
```

```
    using SubscriberId = std::string;
```

```
    using u64 = std::uint64_t;    You, 4 days ago • PublishSubscriber
```

```
    void add_capability(VehicleDataType type) { capabilities_.insert(type); }
```

```
    void remove_capability(VehicleDataType type) { capabilities_.erase(type); }
```

```
    // tell what data types this component can produce
```

```
    bool supports(VehicleDataType type) const { return capabilities_.count(type) > 0; }
```

```
    // someone wants to subscribe to a data type
```

```
    // who = subscriber origin in format of ADDRESS:PORT
```

```
    bool subscribe(VehicleDataType type, const SubscriberId& who, u64 period_us) {
```

```
        if (!supports(type) || period_us == 0) return false;
```

Dados de latência

```
latency_reports >  summary_latency_per_vm.csv
```

1	recv_vm,recv_port,count,avg_us,median_us,std_us
2	2,102,35,"1,681.46","1,016.00","3,420.46"
3	2,65535,209,"30,674.88","10,987.00","178,363.40"
4	3,102,35,"2,168.46","1,018.00","5,867.64"
5	3,65535,209,"290,065.60","319,763.00","219,854.39"
6	4,102,1,"4,506.00","4,506.00",0.00
7	4,65535,38,"-13,013.42","1,840.50","93,729.25"
8	

Conclusões

- De forma geral, a equipe conseguiu concluir as tarefas propostas para a quarta entrega do projeto de Comunicação em Sistemas Autônomos Críticos.
 - Foi possível implementar publish-subscribe, além de aplicar melhorias no projeto como um todo.
-